

Respostas Escritas

Estrutura dos módulos

```
.
├── main.py
├── requirements.txt
├── README.md
├── docs/
│   └── respostas.md
├── src/
│   ├── config.py
│   ├── payment.py
│   ├── channel.py
│   ├── service.py
│   ├── kiosk.py
│   └── giftcard.py
└── tests/
    ├── test_1.py
    ├── test_2.py
    ├── test_3.py
    ├── test_4.py
    ├── test_5.py
    ├── test_6.py
    ├── test_7.py
    └── test_8.py
```

A pasta src tem um arquivo por assunto. O config.py ficou com o AppConfig e também com o Produto, o Order e o OrderBuilder, que são as peças que todo o resto usa como dado de entrada. Formas de pagamento, processors e o registro delas ficaram no payment.py, e o equivalente do lado do canal (checkout, notificação, fábricas e registro) ficou no channel.py. Sobraram pro service.py o OrderService e o EventLogger.

O kiosk.py e o giftcard.py estão separados de propósito. Os dois são extensões que entraram depois, e a graça do exercício era conseguir adicionar canal e forma de pagamento novos sem abrir os arquivos originais, então botar cada um no seu arquivo é o que mostra isso na prática. Se eles estivessem dentro do channel.py e do payment.py, funcionaria igual, mas aí não dava pra provar que os arquivos antigos ficaram intactos.

O main.py na raiz é a inicialização da aplicação e o exemplo executável ao mesmo tempo. Ele importa os módulos de extensão pra eles se registrarem, pega o AppConfig, monta o pedido pelo builder e roda o fluxo nos três canais. Os testes ficam em tests, um arquivo por questão, o que deixa fácil conferir o que cada questão pediu.

Questão 1

1.1. Cada vez que tentássemos criar um objeto da classe e o `__new__` fosse chamado, logo em seguida, o `__init__` também seria chamado. Então, mesmo que na prática não tenha sido criado um objeto novo, ele pode inicializá-lo de novo por cima do que já estava lá.

1.2. O Python só executa um arquivo importado na primeira vez. Nas importações seguintes, ele reaproveita o que já está na memória. Logo, variáveis globais em um arquivo `.py` funcionam como um Singleton nativo.

1.3. Qualquer parte do código pode alterar os dados de forma invisível. Isso causa bugs difíceis de rastrear e faz com que os testes interfiram uns nos outros.

Questão 2

2.1. A classe `OrderBuilder` atua como o Builder e a classe `Order` é o objeto final construído.

2.2. O construtor direto funciona no Python por causa dos parâmetros nomeados e opcionais. A vantagem do Builder é permitir a construção em etapas, enquanto o construtor exige todos os dados de uma vez só na hora de instanciar.

Questão 3

3.1. O Creator é o `PaymentProcessor`, que define o fluxo `process_order` sem saber qual pagamento vai usar. Cada processor concreto (`PixProcessor`, `CreditCardProcessor`, `BoletoProcessor`) é o Concrete Creator, porque implementa o `create_payment()` decidindo qual classe instanciar. O Product é a classe abstrata `Payment`, com o método `pay`, e o Concrete Product é cada implementação dela: `PixPayment`, `CreditCardPayment` e `BoletoPayment`.

3.2. Um `if/elif` escolhendo a classe concreta deixa essa decisão espalhada em qualquer lugar do código que precisa criar um pagamento, e toda forma nova exige mexer nesse trecho. O Factory Method separa essa escolha, colocando ela dentro de um método que cada subclasse do Creator implementa por conta própria. Assim quem usa o processor, no caso o `process_order`, nunca fica sabendo qual pagamento concreto foi criado.

3.3. Precisa criar a classe nova de `Payment` com o método `pay`, o Processor correspondente com o `create_payment` dela, e uma linha registrando essa forma no dicionário que o `get_payment_processor` consulta. O `PaymentProcessor` e o `process_order` continuam iguais, porque nenhum dos dois conhece as classes concretas de pagamento. Foi exatamente esse caminho que segui na questão 8 pra adicionar o gift card.

Questão 4

4.1. Checkout e Notification formam uma família porque cada canal precisa de uma versão compatível dos dois. O canal Web lista os produtos um por um e avisa o cliente por e-mail, enquanto o Mobile mostra só o resumo e manda push. Cada par desses segue a mesma convenção do canal onde nasceu, por isso misturar o checkout de um canal com a notificação de outro não faz sentido dentro da loja.

4.2. A Abstract Factory resolve o problema de garantir que checkout e notificação combinam entre si, sem que o código que usa a fábrica precise saber qual canal está ativo. Esse código só chama `create_checkout()` e `create_notification()` da fábrica que recebeu, então não corre o risco de montar um checkout de um canal com a notificação de outro, porque isso fica garantido dentro da própria fábrica.

4.3. Porque canal e forma de pagamento variam de forma independente, o próprio enunciado já deixa isso claro dizendo que WEB pode usar PIX e MOBILE pode usar cartão. Se o pagamento estivesse dentro da ChannelFactory, cada combinação de canal com forma de pagamento ia precisar da própria fábrica, tipo uma WebPixFactory e uma WebCartaoFactory, e isso cresce rápido demais. Deixando o pagamento de fora, a fábrica do canal cuida só do que realmente é responsabilidade dela.

Questão 5

5.1. Pra adicionar o KIOSK criei um arquivo novo, o `src/kiosk.py`, com `KioskCheckout`, `KioskNotification` e `KioskFactory`, e no final dele chamei o registro dessa fábrica no dicionário que o `get_channel_factory` usa. A única coisa fora desse arquivo é a linha de import do `src.kiosk` na inicialização da aplicação, que é o que faz o registro rodar. O `src/channel.py` e todo trecho que já usava `Checkout` ou `Notification` ficaram intactos.

5.2. É compatível com o OCP porque a extensão inteira foi um arquivo novo, sem tocar em código já existente e testado. O `get_channel_factory` e o resto do sistema continuam trabalhando só com as abstrações `ChannelFactory`, `Checkout` e `Notification`, então nem precisam saber que o KIOSK existe, só enxergam mais uma fábrica registrada.

Questão 6

6.1. O `AppConfig` guarda a configuração compartilhada da aplicação. O `OrderBuilder` monta o pedido aos poucos e o `Order` guarda os dados dele e calcula o total. Quem decide qual pagamento usar e executa ele é o `PaymentProcessor` com as subclasses de pagamento. Já o `ChannelFactory` e as subclasses de canal (`Web`, `Mobile`, `Kiosk`) criam o checkout e a notificação certos pra cada canal, sendo que o `Checkout` mostra o resumo do pedido pro cliente e o `Notification` avisa que o pedido foi processado. O `EventLogger` fica só com o registro de que o pedido passou pelo fluxo, e o `OrderService` fica no meio de tudo isso coordenando a ordem das chamadas, sem fazer o trabalho de nenhum desses colaboradores sozinho. Fora essas classes, o `main.py` tem a responsabilidade de montar tudo, é ele que decide qual processor e qual fábrica o `OrderService` vai receber naquela execução.

6.2. Trocar como a notificação é enviada, tipo email por SMS, fica restrito à classe de `Notification` do canal. A regra de cálculo do total, por exemplo aplicar desconto de cupom, é uma mudança que só deveria mexer no método `total()` de `Order`. E se o log precisar ir pra um arquivo em vez de aparecer só no console, isso é problema exclusivo do `EventLogger`, o resto do sistema nem percebe a troca.

6.3. Uma decisão que dava pra ser diferente é registrar os canais usando string (`WEB`, `MOBILE`, `KIOSK`) em vez de um Enum do Python. Com Enum, um erro de digitação no nome do canal apareceria na hora de rodar o código, em vez de só quando o `get_channel_factory` não encontrasse a fábrica. Mas usando string fica mais fácil registrar um canal novo vindo de fora do código, tipo de um arquivo de configuração, sem precisar editar uma classe de Enum toda vez.

Questão 7

7.1. O primeiro teste zera o `_instance` na mão e pede uma configuração nova, pra conferir quais valores o Singleton assume quando é criado do zero. O esperado é nascer com `environment` em `production`, `currency` em `BRL` e `debug` desligado. Os testes da questão 1 só olhavam se o objeto era compartilhado e se as alterações grudavam, então esse aqui cobre a outra ponta, que é a aplicação subir num estado inicial conhecido em vez de depender do que outro teste deixou pra trás.

7.2. O segundo teste constrói um pedido só com cliente, sem nenhum produto, e olha o que acontece com o total. O esperado é a lista de produtos ficar vazia e o `total()` devolver zero, sem estourar erro. O `build()` só exige cliente, então pedido sem produto é um estado que o builder aceita de propósito. Se o `total()` não aguentasse esse caso, o sistema quebraria num pedido que ele mesmo considera válido, que é o tipo de inconsistência chata de achar depois.

7.3. O terceiro teste pega o checkout da `WebFactory` e o da `MobileFactory` e compara os dois. O esperado é cada fábrica devolver uma classe concreta diferente, `WebCheckout` de um lado e `MobileCheckout` do outro. Esse comportamento importa porque é o que garante que a fábrica está mesmo trocando a implementação por canal. Se as duas devolvessem a mesma coisa, o código continuaria rodando sem erro nenhum, só que o canal teria virado enfeite e a `Abstract Factory` não estaria fazendo trabalho nenhum.

Questão 8

8.1. A forma de pagamento escolhida foi o gift card. Criei dois arquivos, o `src/giftcard.py` com o `GiftCardPayment` e o `GiftCardProcessor`, e o `tests/test_8.py` com os testes dela. No `main.py` entrou uma linha de `import` do `src.giftcard`, que é o que faz o módulo carregar e se registrar sozinho quando a aplicação sobe. Fora essa linha de `import`, nenhum arquivo existente foi tocado.

8.2. Não. O `process(order)` do `OrderService` e o `process_order(order)` do `PaymentProcessor` continuam exatamente como estavam, porque os dois só mexem com as abstrações e nunca citam classe concreta de pagamento. Quem resolve qual processor usar é o `get_payment_processor`, que consulta o dicionário de formas registradas e devolve o que estiver lá, sem se importar com quem colocou.

8.3. Nenhuma. O `GiftCardPayment` herda de `Payment` e o `GiftCardProcessor` herda de `PaymentProcessor`, então os dois entram no sistema por herança e pelo registro, sem precisar abrir nenhuma das classes que já existiam.

8.4. O `Factory Method` já tinha isolado a criação do objeto de pagamento dentro do `create_payment()`. Como o fluxo comum mora no `process_order` e ele só chama `create_payment()` sem olhar o que volta, bastou escrever uma subclasse de `PaymentProcessor` devolvendo `GiftCardPayment` pra forma nova entrar inteira no sistema. Se a escolha da classe concreta estivesse espalhada em `if/elif` pelo código, cada um desses pontos teria que ser encontrado e alterado.

8.5. As duas extensões seguem o mesmo desenho: um arquivo novo com as classes concretas, uma chamada de registro no fim dele e um `import` na inicialização da aplicação. Nos dois casos o fluxo principal ficou intacto e nenhuma classe existente precisou ser editada. A

diferença está no que cada padrão precisa criar. O Factory Method do gift card entrega um produto só, o Payment, então uma subclasse de PaymentProcessor dá conta. A Abstract Factory do KIOSK entrega uma família, Checkout e Notification, e a KioskFactory tem que garantir que as duas peças combinam entre si. Os dois também variam por motivos diferentes, a forma de pagamento muda de pedido pra pedido e o canal muda conforme a venda entra pela web, pelo celular ou pelo totem, e é por isso que cada um tem o seu registro em vez de um registro só tentando dar conta das duas coisas.